Django – Full Stack Framework

MVC Architecture

- MVC (Model-View-Controller) is a software architectural pattern that separates an application into three interconnected components: **Model** (data/logic), **View** (UI), and **Controller** (intermediary).
- This separation ensures clean organization, allowing developers to modify the UI or backend logic independently

MVC Architecture

- **The Three Core Components**
- **Model:** Manages the data and core business logic. It holds the state of the application, interacts directly with the database, and notifies the view (or controller) when data changes.
- **View:** Handles the layout, design, and user interface. It displays the information retrieved from the Model and captures user interactions to send to the Controller.
- **Controller:** Acts as the "brain" or middleman. It receives user input from the **View**, processes application logic, updates the **Model**, and ultimately selects the appropriate **View** to render the final response.

MVC Architecture

How the Flow Works

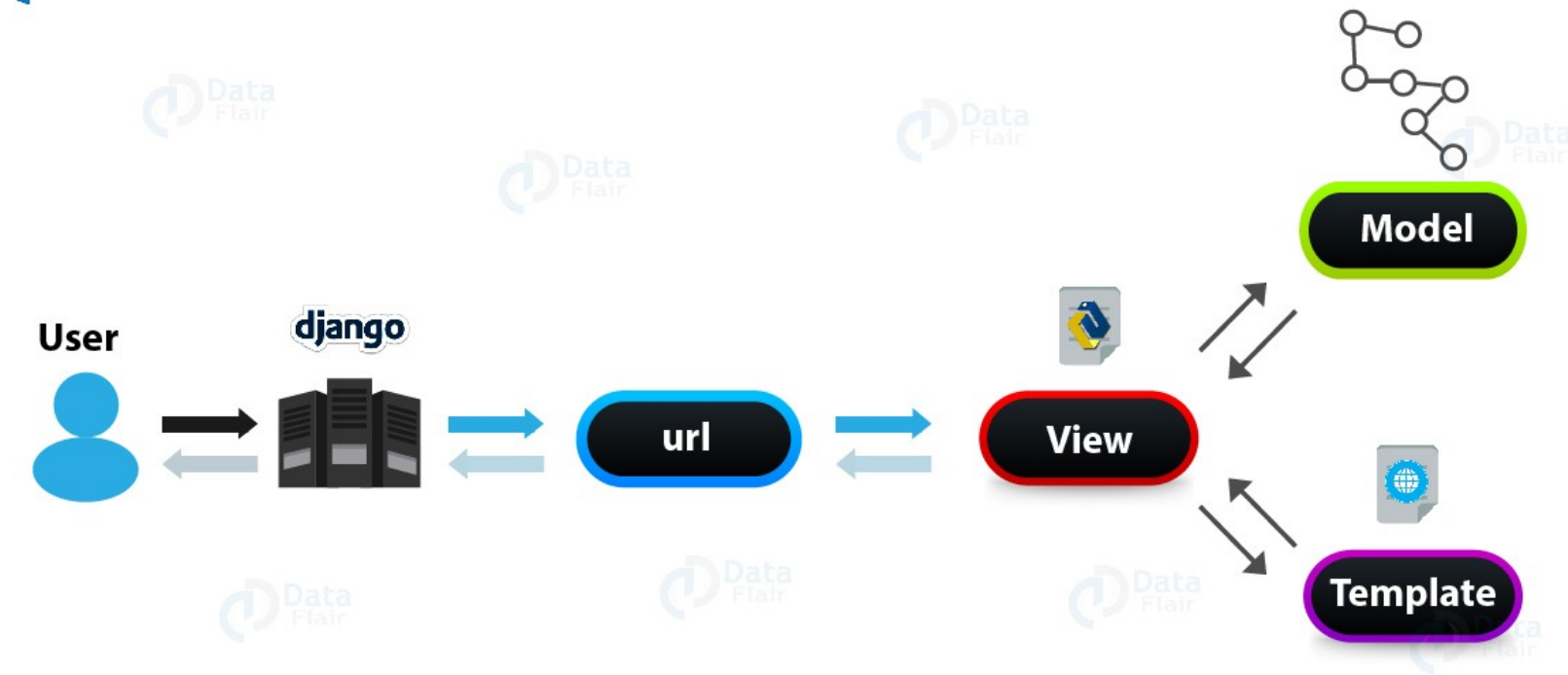
- A user interacts with the **View** (e.g., clicking a "Submit" button).
- The **View** sends the user's action/request to the **Controller**.
- The **Controller** interprets the action and calls the **Model** to update or fetch data.
- The **Model** performs the necessary business logic, queries the database, and returns the updated state to the **Controller**.
- The **Controller** passes this data to the correct **View**.
- The **View** updates and renders the new state to the user

Django – MVT/MTV

- In Django, the classic MVC (Model-View-Controller) design pattern is implemented a little uniquely.
- Django’s creators actually describe its architecture as **MVT (Model-View-Template)**.

The MVC to Django Mapping

Traditional MVC Component	Django's Equivalent	What it does in Django
Model	**M**odel	Definement of your database tables using Python classes (`models.py`).
View	**T**emplate	The actual layout design, written in HTML and CSS (`.html` files).
Controller	**V**iew	The core Python functions containing your business logic (`views.py`).


Data Flair
User
django
url
View
Model
Template

1. The Model (Data Layer)

- Just like in traditional MVC, the **Model** is responsible for managing data.
- Instead of writing raw SQL commands to create tables or extract rows, you write standard Python classes.
- Django converts those classes into database tables automatically(ORM).
- *Where it lives:* models.py
- *Analogy:* The file cabinet or blueprint for your information.

2. The Template (The Presentation Layer — Traditional "View")

- In traditional software terms, the "View" is what the user *sees*.
- In Django, this is called the **Template**.
- It consists of HTML files embedded with **Django Template Language (DTL)** tags (like {% for project in projects %}).
- This allows Python to insert dynamic data cleanly into your web page's static visual elements.
- *Where it lives:* Inside your app's templates/ folder.
- *Analogy:* The blank document form waiting to be filled out with client details.

3. The View (The Logic Layer — Traditional "Controller")

- In traditional MVC, the **Controller** acts as the brain—it takes user input, talks to the model, processes logic, and decides what to show. In Django, this brain function lives inside the **View**.
- *Where it lives:* views.py
- *Analogy:* The restaurant waiter who takes your order, carries it to the kitchen (Model), gets your food, plates it up beautifully, and presents it to your table (Template).

(The Request – Response Cycle)

- When a user visits your website, Django passes the request through these layers in a precise loop:
- **The URL Router (urls.py):** The user types in an address. Django’s routing configuration checks the pattern and directs the request to the matching function inside your **View**.
- **The View Execution (views.py):** The View runs your Python logic.
- **The Model Request (models.py):** The View reaches out to your **Model** to query the database. The Model pulls the correct data rows from your SQLite/PostgreSQL storage and hands them back to the View.
- **The Template Synthesis (home.html):** The View takes that raw database entry and hands it off to your HTML **Template**.
- **The Response Delivery:** Django stitches the dynamic data directly into the HTML format and ships a clean, rendered webpage straight back to the user's browser view.

URL Routing

- URL routing is Django's mechanism for mapping browser requests (URLs) to specific Python code execution blocks (Views).
- In Django, the URL router acts as the **front desk receptionist** of your web application. When a user enters a web address into their browser, the URL router instantly reads the request and determines which Python function (the View) should process it.
- Every Django project maps these paths using list objects called **urlpatterns** within specialized Python configuration files named urls.py.

How URL Routing Works in Django

User

User enters URL in browser
http://127.0.0.1:8000/about/

User requests a web page by entering a URL.

Browser Request

Browser sends HTTP request to the server

The browser sends the request to Django server.

Project urls.py

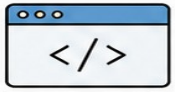
Django looks into project urls.py
(Root URL Configuration)

The root urls.py receives the request and decides which application should handle it.

App urls.py

Django looks into app urls.py
(URL Pattern Matching)

The app's urls.py matches the URL pattern and selects the appropriate view function.



View Function (views.py)

View function executes
(Contains business logic)

The corresponding view function is executed to process the request.

Template (HTML)

View renders the template
(HTML page is generated)

The view sends data to the template and the HTML page is generated.

Response to Browser

Django sends HTTP response to browser

The generated page is sent back to the browser and displayed to the user.

- urlpatterns is just a **standard Python list** that you write inside a urls.py file.
- Django reads this list from top to bottom every single time someone types an address into their web browser to find out where to send them.

How the Mapping Works

- A standard Django web address map is created using the built-in **path()** function, which takes three primary arguments:

  Python

❖ path(route, view_function, name)

- **route**: The text string matching the URL path typed into the browser bar (e.g., 'contact/'). An empty string ('') indicates the root homepage directory.
- **view_function**: The specific Python function inside your views.py file that contains the processing logic to execute when this URL path is requested.

- **name**: A unique text label assigned to the route. This allowsyou to link or redirect to this path anywhere else in yourproject using its name instead of hard coding raw paths like/my-app/dashboard/v2/.

What is `urls.py`?

The **`urls.py`** file contains **URL patterns** that map URLs to view functions.

For example:

```
from django.urls import path from . import views
urlpatterns =
 [ path('', views.home),
 path('about/', views.about),
 path('contact/', views.contact), ]
```

What is Views.py?

The **view** contains the business logic and returns a response to the user.

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome Home")

def about(request):
    return HttpResponse("About Us")

def contact(request):
    return HttpResponse("Contact Page")
```

Browser URL	Django View
`/`	home()
`/about/`	about()
`/contact/`	contact()

URL Pattern Syntax:

path('URL/', view_function)

Example:

path('students/', views.students)

Dynamic Routing

Django allows variables to be passed through the URL.

Imagine an online store with **10,000 different products**. You would never write 10,000 static URL lines. Instead, you write **one single dynamic path** that acts as a catch-all variable catcher.

❖ **The App-Level Code (products/urls.py):**

urlpatterns = [path('item/<int:product_id>/', views.product_detail_view, name='product_detail'),]

The Matching View Logic (products/views.py):

```
from .models import Product
# Django grabs that number from the URL and automatically forces it into this function argument!
def product_detail_view(request, product_id):
    # If the URL was /item/582/, product_id becomes 582
    item = Product.objects.get(id=product_id)
    return render(request, 'detail.html', {'item': item})
```

If the user visits:

http://127.0.0.1:8000/product/582/

Output:

```
product ID = 582
```

Inside your single template file (detail.html):

```
<!-- This single file handles EVERY product -->
<html>
<head>
    <title>{{ item.name }}</title>
</head>
<body>
    <h1>{{ item.name }}</h1>
    <p>Price: ${{ item.price }}</p>
    <p>Description: {{ item.description }}</p>
</body>
</html>
```

How Django swaps the data based on the Product ID:

Scenario A: User visits /item/1/

The View looks up Product #1 in the database: {"name": "Wireless Mouse", "price": 25}.

Django passes this data to detail.html.

The user sees a webpage titled **Wireless Mouse** with a price of **$25**.

Scenario B: User visits /item/2/

The View looks up Product #2 in the database: {"name": "Mechanical Keyboard", "price": 80}.

Django passes this data to the *exact same* detail.html file.

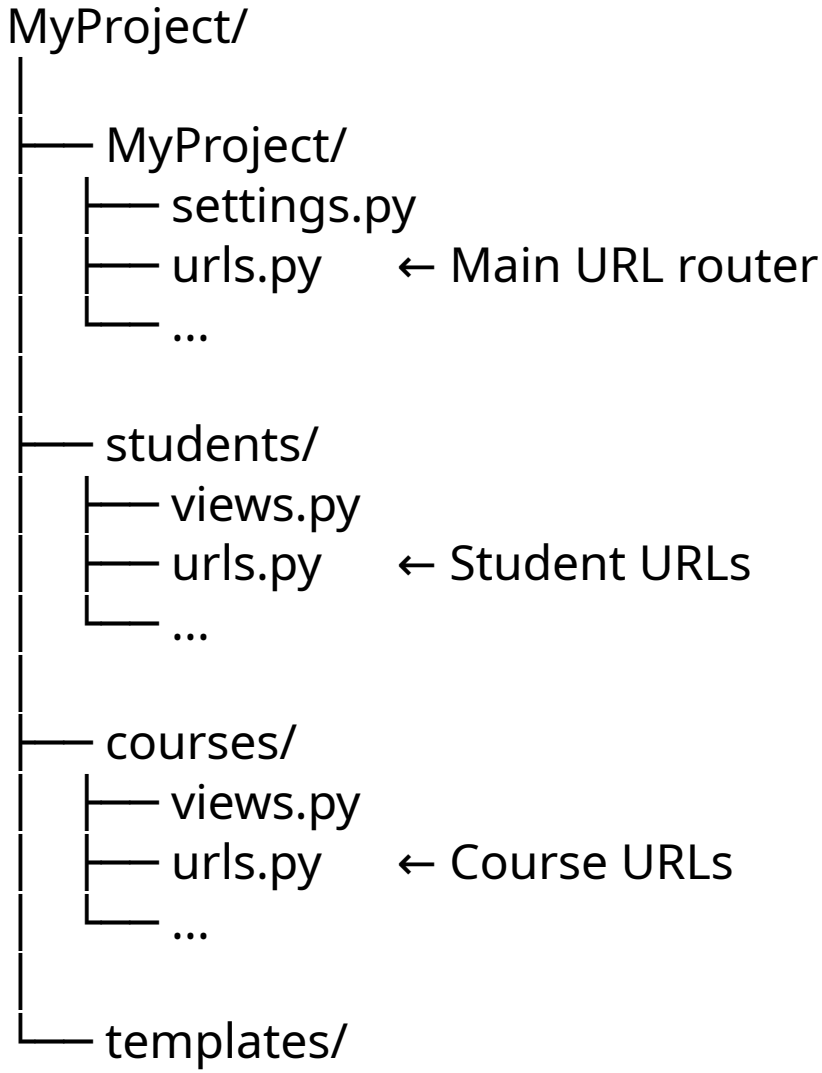
The placeholder {{ item.name }} instantly updates, and the user sees **Mechanical Keyboard** with a price of**$80**.

Decoupled Routing
(The Multi-App Project Model)

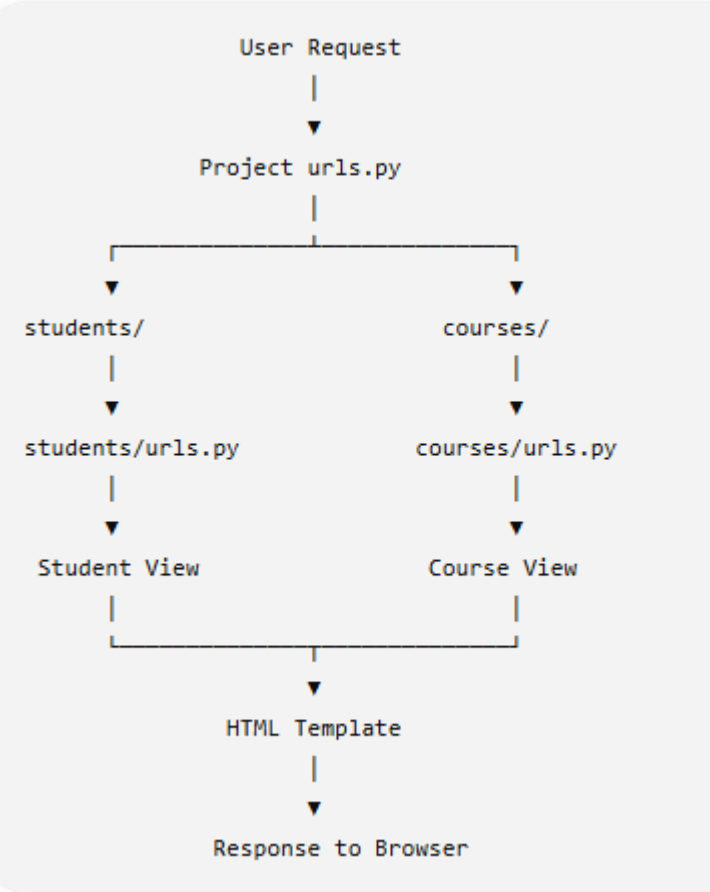
In **Django**, **Decoupled Routing** means that the **URL patterns are defined separately from the view functions**. The URL dispatcher (`urls.py`) is responsible for mapping a URL to a view, while the view (`views.py`) contains the application logic.

This separation follows the **Separation of Concerns (SoC)** principle, making Django applications clean, modular, and easy to maintain.

Imagine a complex university portal system that has a **Student Management** section and a separate **Online Library** section. Instead of jamming all the URLs into one master file, you decouple them by splitting them into separate files.


MyProject/
MyProject/
settings.py
urls.py ← Main URL router
...
students/
views.py
urls.py ← Student URLs
...
courses/
views.py
urls.py ← Course URLs
...
templates/

How Decoupled Routing Works



The Master Project Router (university_project/urls.py)

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),

    # If it starts with 'students/', throw it over to the student app
    path('students/', include('student_portal.urls')),

    # If it starts with 'library/', throw it over to the library app
    path('library/', include('library_catalog.urls')),
]
```

The Independent App Routers (Decoupled Files)

Now, each app handles its own internal routing configuration locally inside its own folder completely independent of the other.

Inside the Student Portal app (student_portal/urls.py):

```
from django.urls import path
from . import views
urlpatterns = [
    path('dashboard/', views.student_dashboard), # Full URL: /students/dashboard/
    path('grades/', views.view_grades),          # Full URL: /students/grades/
]
```

Inside the Library Catalog app (library_catalog/urls.py):

```
from django.urls import path
from . import views
urlpatterns = [
    path('search/', views.book_search),          # Full URL: /library/search/
    path('renew/', views.renew_books),           # Full URL: /library/renew/
]
```

Advantages of URL Routing

- Maps URLs to the correct view functions.
- Makes applications organized and easy to maintain.
- Supports dynamic URLs with parameters.
- Enables modular application development using `include()`.
- Improves readability and scalability of large projects.

1.Function-Based Views (FBVs)

This reminds you how to route to a regular Python **function**.

The Code Example:

Python

```
from my_app import views path('', views.home, name='home')
```

What it means: You wrote a simple function named def home(request): inside your views.py file. This layout tells Django: *"When a user visits the homepage (''), run that specific home function."*

2. Class-Based Views (CBVs)

This reminds you how to route to a Python **class** instead of a function. As projects grow, Django lets you design views using classes because they allow you to reuse code easily.

The Code Example:

Python

```
from other_app.views import Home
path('', Home.as_view(), name='home')
```

What it means: You created a class named class Home(View): instead of a function.Because Django's URL routing engine *only* knows how to talk to functions, you must append **.as_view()** to the end of the class name. This magic internal method temporarily transforms your class layout into a function so the router can process it.

View in Django

- The view is a Python function (or class) that contains the actual core brain logic of your application.
- It takes an incoming web request from a user, processes data, interacts with your database, and sends a response back to the browser.

What a View Actually Does (The 3-Step Lifecycle)

[Incoming Request] ── 1. Read Request ── 2. Process Logic / DB ── 3. Return Response

Receives an HttpRequest object: This object contains metadata about the user's action (e.g., what URL they visited, data they submitted in a form, or their login credentials).

Executes processing logic: This is where you write your Python code. It can run a machine learning pipeline, calculate a utility score, or run a query against a database.

Returns an HttpResponse object: A view *must* return a response. This can be a rendered HTML webpage, a redirect to another page, or raw data strings.

The Two Ways to Return Data from a View

Depending on whether you are building a traditional website or a headless data engine, your view will return data differently.

Method A: Returning Web Pages (Traditional MVT Setup)

If your backend is responsible for creating visual web pages, the view uses the **render()** helper function to stitch data into an HTML template file.

```
# projects/views.py
from django.shortcuts import render
from .models import Device


def device_dashboard_view(request):
    # 1. Fetch data rows from the database
    all_devices = Device.objects.all()

    # 2. Package data and stitch it directly into an HTML file
    return render(request, 'dashboard.html', {'devices': all_devices})
```

Method B: Returning Raw Data (Headless Backend Setup)

If you are building a headless system to pass raw information directly to a mobile app framework or a JavaScript frontend, the view bypasses HTML entirely and returns a **JsonResponse**.

```
# api/views.py
from django.http import JsonResponse
from .models import Device


def device_list_api(request):
    # 1. Fetch data rows from the database
    all_devices = Device.objects.all().values('id', 'status', 'battery_level')

    # 2. Convert the database records into a standard Python list
    data_list = list(all_devices)

    # 3. Ship raw JSON text instantly across the network
    return JsonResponse({"devices": data_list}, safe=False)
```

Templates in Django

- In Django, **Templates** are the third pillar of the MVT (Model-View-Template) architecture. They handle the user interface (UI) and layout of your website.
- Think of a Django template as a **smart HTML file**. It contains standard HTML code for structure and styling, mixed with special placeholder tags that allow Django to inject dynamic data from your database on the fly.

The Magic Syntax: Django Template Language (DTL)

Django doesn't just send static HTML to the browser. It uses its own built-in templating language to make HTML dynamic.

There are three core syntax elements you will use constantly:

Variables: {{ variable }} (The Placeholders)

Used when you want to print out a piece of data sent from your view (like a blog post title or an author's name).

<h1>{{ post.title }}</h1>

Written by: {{ post.author }}

Tags: {% tag %} (The Logic)

- Used for logic processing, like looping through a list of items or checking a condition.

```
<!-- Looping through all your projects -->
{% for project in project_list %}
    <h3>{{ project.title }}</h3>
{% empty %}
    <p>No projects uploaded yet!</p>
{% endfor %}
```

Filters: {{ variable|filter }} (The Formatters)

- Used to modify how a variable looks right inside the HTML layout before displaying it.

<!-- Capitalizes the text and formats a date cleanly -->

{{ user.name|title }}

Published on: {{ post.date_published|date:"F j, Y" }}

Advanced Template Feature: Template Inheritance

- When building a blog or portfolio, you don't want to copy and paste your navigation bar, footer, and CSS links onto every single HTML page. Django fixes this with **Template Inheritance**.
- You create one master "parent" layout file, and all other "child" files inherit from it and only fill in their unique content blocks.

Step 1: The Parent Layout (base.html)
This file defines the shared skeleton framework of your entire website.

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <nav>
        <a href="/">Home</a> | <a href="/blog/">Blog</a>
    </nav>

    <!-- This is a blank container for child files to inject into -->
    <div class="content">
        {% block content %}
        {% endblock %}
    </div>

    <footer>© 2026 My Portfolio Site</footer>
</body>
</html>
```

- **The Child Layout (blog_list.html)**

This file inherits the base framework and only worries about its specific content.

```
<!-- templates/blog_list.html -->
{% extends 'base.html' %}  <!-- Ties this file back to the master skeleton -->


{% block content %}
    <h2>Latest Blog Posts</h2>
    <p>Welcome to my blog! Here are my thoughts:</p>
    <!-- Your loop logic goes here -->
{% endblock %}
```

How the View Connects to the Template

To send data to your template, your view wraps your database info inside a Python dictionary called a **Context** and hands it over to the render() function.

```
# blog/views.py
from django.shortcuts import render
from .models import Post


def blog_home(request):
    posts = Post.objects.all()

    # Context dictionary: The left side is the name your HTML file will read,
    # the right side is the actual Python variable containing your data.
    context = {'project_list': posts}

    return render(request, 'blog_list.html', context)
```

When a user visits the page, Django merges blog_list.html, fills in the {% block content %} slot inside base.html, swaps all the {{ project.title }} loops with actual data, and ships a standard HTML file back to the user's browser.

Database Integration Using Django ORM

What is an ORM?

- **Definition**: **Object-Relational Mapping (ORM)** is a programming technique that connects a relational database (like PostgreSQL, MySQL, SQLite) to an object-oriented programming language (like Python).
- **How it works**: It maps database tables into Python classes (called **Models**) and table rows/columns into Python objects and attributes.
- **The Benefit**: Developers can create, read, update, and delete database records using pure Python syntax **without writing raw SQL queries**.

Django Models & Fields

In Django, every database table is defined as a Python class in the models.py file. Each class attribute represents a column in the table.

Example: Creating a Model

```python
from django.db import models


class Student(models.Model):
  name = models.CharField(max_length=100)
  roll_number = models.IntegerField(unique=True)
  email = models.EmailField()
  admission_date = models.DateField(auto_now_add=True)

  def __str__(self):
    return self.name
```

Common Field Types:

- models.CharField(max_length=...): For short strings/text.
- models.TextField(): For long blocks of text.
- models.IntegerField() / FloatField(): For numbers.
- models.DateField() / DateTimeField(): For dates and times.
- models.EmailField(): For email validation.

Database Migrations (Version Control for Databases)

•Once models are written, Django needs to translate them into actual database tables. This is done via **Migrations**.

•**`python manage.py makemigrations`**: Inspects `models.py` and generates migration script files (blueprints) in the `migrations/` folder.

•**`python manage.py migrate`**: Executes those scripts against the database, creating or altering the physical tables.

CRUD Operations Using QuerySets

A **QuerySet** is a collection of database queries or objects fetched from your database. You can interact with them directly using Django's interactive shell (python manage.py shell).

A. Creating (Inserting) Data

There are two primary ways to insert a record:

```
from myapp.models import Student

# Method 1: Instantiate and save
s = Student(name="Alice", roll_number=101, email="alice@example.com")
s.save()

# Method 2: Using the create() shortcut
Student.objects.create(name="Bob", roll_number=102, email="bob@example.com")
```

B. Reading (Retrieving) Data

- **Get all records**:

```
students = Student.objects.all()
print(students)
```

- **Filter records (Condition matching)**:

```
# Find students named Alice
results = Student.objects.filter(name="Alice")
```

- **Get a single specific object**:

```
# Returns one unique object (raises error if multiple or none exist)
s = Student.objects.get(roll_number=101)
```

C. Updating Data

To update an existing record, fetch it, change its attributes, and call save():

```
s = Student.objects.get(roll_number=101)
s.email = "alice.new@example.com"
s.save()  # Commits updates to the database
```

D. Deleting Data

Fetch the object and call the .delete() method:

```
s = Student.objects.get(roll_number=102)
s.delete()  # Removes the record from the database
```